

AUTOMOTIVE EMBEDDED & IT SYSTEMS RESEARCH

INDEPENDENT COLLABORATIVE RESEARCH PROJECT

TECHNICAL WHITE PAPER & ARCHITECTURE SPECIFICATION

**RESEARCH AND DEVELOPMENT OF A
SOFTWARE-IN-THE-LOOP DIAGNOSTIC
TESTING FRAMEWORK**

An Automated Virtual ECU Validation Platform Integrating

*DoIP (ISO 13400), UDS (ISO 14229), Simulink Plant Dynamics, and Jenkins
CI/CD*

PROJECT DOMAIN: AUTOMOTIVE SIL TESTING & DEVOPS AUTOMATION

AUTHOR: NGUYEN DINH TUAN & NGUYEN DAN THANH

VERSION / STATUS: VERSION 1.0 - COMPLETED RESEARCH

HO CHI MINH CITY, 2026

ABSTRACT

This report presents the design and verification of a Software-in-the-Loop (SIL) diagnostic testing platform for a virtual Electronic Control Unit (vECU). The platform combines a native C diagnostic core, Diagnostics over Internet Protocol (DoIP), Unified Diagnostic Services (UDS), a MATLAB/Simulink plant model, a C# WPF desktop interface, Tcl-based test automation, and a Jenkins CI/CD pipeline. The implemented prototype provides repeatable validation of routing activation, diagnostic session control, security access, data-identifier operations, diagnostic trouble-code handling, and a simulated Limp Home reaction. By separating the simulation from physical hardware, the framework enables earlier and safer regression testing while preserving execution logs and HTML reports as quality evidence.

List of Abbreviations

AUTOSAR: AUTomotive Open System ARchitecture

CI/CD: Continuous Integration / Continuous Delivery

DCM: Diagnostic Communication Manager

DEM: Diagnostic Event Manager

DID: Data Identifier

DoIP: Diagnostics over Internet Protocol

DTC: Diagnostic Trouble Code

ECU: Electronic Control Unit

FiM: Function Inhibition Manager

HIL: Hardware-in-the-Loop

NvM: Non-Volatile Memory Manager

OSI: Open Systems Interconnection

SIL: Software-in-the-Loop

UDS: Unified Diagnostic Services

vECU: Virtual Electronic Control Unit

VTC: Vehicles Test Case

Contents

1 Introduction.....	4
1.1 Background	8
1.2 Problem Description and Objectives	9
1.3 Method	10
2 Research and Analysis.....	12
2.1 Use Cases for the Simulation Platform.....	12
2.2 DoIP and UDS Architectural Design	12
2.3 Overview of AUTOSAR Diagnostic Stack Architecture.....	21
3 Prototype Development	24
3.1 Overall Software-in-the-Loop Architecture.....	24
3.2 Session Management and Security Access State Machine	25
3.3 Simulink Co-Simulation and Limp Home Safety Logic	26
3.4 Automated Testing Framework and Jenkins CI/CD Pipeline	28
4 Testing, Results, and Evaluation	30
4.1 Verification and Test Execution	30
4.2 Automated CI/CD Regression Results.....	38
5 Conclusion and Future Work	40
5.1 Conclusion	40
5.2 Future Work	40
References	41

List of Figures

1 Introduction.....	3
Figure 1.1: Vehicle Electrical and Electronic (E/E) Architecture	9
Figure 1.2: vECU Software-in-the-Loop Diagnostic Testing Ecosystem.....	10
2 Research and Analysis.....	6
Figure 2.1: 7-Layer OSI Model vs UDS ISO Standards.....	13
Figure 2.2: DoIP Discovery, Vehicle Identification & Network Topology.....	13
Figure 2.3: DoIP Gateway Topology & Socket Connection Instance Model	21
Figure 2.4: Dependencies to other modules (AUTOSAR BSW Architecture).....	22
Figure 2.5: DEM Event Memory & FreezeFrame Record Structure.....	23
3 Prototype Development.....	12
Figure 3.1: Overall SIL Platform Technical Architecture	24
Figure 3.2: Session and Security State Machine.....	25
Figure 3.3: UDS Diagnostic & Security Access Execution Sequence.....	27
Figure 3.4: MATLAB/Simulink Closed-Loop Powertrain Plant Model & FiM Limp- Home Logic	28
Figure 3.5: Jenkins Automated CI/CD Regression Pipeline.....	29
4 Testing, Results, and Evaluation	18
Figure 4.1: vECU C-Core Execution and DoIP Server Startup Terminal Log	30
Figure 4.2: TCP Socket Connection Acceptance and Disconnection Log.....	31
Figure 4.3: MATLAB/Simulink Powertrain Model - Normal Operating State	31
Figure 4.4: MATLAB/Simulink Powertrain Model - Coolant Leakage Fault Injection Setup	32
Figure 4.5: Dynamic Co-Simulation Data Transfer Log from Simulink to vECU RAM via UDS 0x2E	33
Figure 4.6: DoIP/UDS Data Processing Flow Log - Read Vehicle Speed (DID 0x0100).....	34
Figure 4.7: DoIP/UDS Data Processing Flow Log - Read Engine RPM (DID 0x0101)....	35
Figure 4.8: DoIP/UDS Data Processing Flow Log - Read Normal Coolant Temperature (DID 0x0102).....	35
Figure 4.9: DoIP/UDS Fault Injection Log - Write Overheat Temperature (183°C) and DTC P0115	36

Figure 4.10: C# WPF Test Automation Dashboard - Successful Test Suite Execution (4/4 Passed)	37
Figure 4.11: C# WPF Test Automation Dashboard - Assertion Failure Detection on Overheat Condition.....	37
Figure 4.12: Automated UDS HTML Test Execution Report - 100% Passed	38
Figure 4.13: Automated UDS HTML Test Execution Report - Failed Execution Summary	39

List of Tables

Table 2.1: Summary of Unified Diagnostic Services (UDS SIDs) Implemented in the vECU Platform.....	16
Table 2.2: Supported Diagnostic Data Identifiers (DIDs) in the vECU Diagnostic Stack	18
Table 2.3: Implemented Negative Response Codes (NRCs) Matrix in vECU Diagnostic Stack.....	19

CHAPTER 1. INTRODUCTION

1.1 Background

Nowadays, modern vehicles are no longer purely mechanical entities; they are highly complex systems containing a large number of onboard computers. These computers, widely known as Electronic Control Units (ECUs), are responsible for both providing services to the driver and performing fine-grained engine mechanisms. As the complexity of vehicle electronics has exponentially increased, driven by electrification and advanced driver-assistance systems, diagnostic applications must continually adapt to support evolving services, complex networks, and large data transfers. Traditionally, diagnostic procedures required physical access to the vehicle's hardware via standard connectors. However, relying exclusively on physical hardware for testing and development presents significant challenges because it is expensive, inflexible, and makes unsafe fault scenarios difficult and hazardous to reproduce safely. To overcome these challenges, a virtual ECU (vECU) operating within a Software-in-the-Loop (SIL) environment provides an accessible, earlier, and safer validation point. Furthermore, the evolution of in-vehicle networks towards Ethernet-based systems has introduced Diagnostics over Internet Protocol (DoIP), which enables high-bandwidth communication and shifts the paradigm from traditional bus systems to IP-based networks.

Vehicle Electrical and Electronic (E/E) Architecture

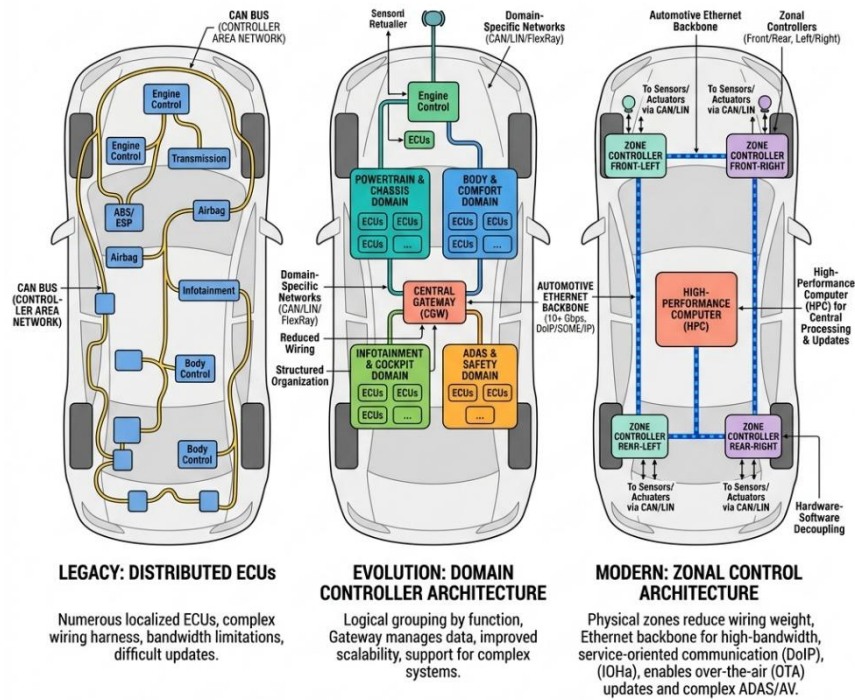


Figure 1.1: Vehicle Electrical and Electronic (E/E) Architecture

Evolution from Distributed ECUs to Domain Controllers and Modern Zonal DoIP Architecture.

1.2 Problem Description and Objectives

The core question this independent research project strives to answer is how an automated Software-in-the-Loop diagnostic testing framework can be designed to seamlessly integrate an engine-control virtual ECU with modern Continuous Integration and Continuous Delivery (CI/CD) pipelines. To answer this question, the research addresses two interconnected problems. First, it requires the implementation of a diagnostic vECU featuring a native C core that successfully exposes DoIP and Unified Diagnostic Services (UDS) interfaces. Second, the platform must automate the execution of regression tests through scripting technologies without manual intervention. The objective of this collaborative framework is to provide a highly repeatable diagnostic test environment. By separating the simulation from physical hardware, the system allows engineers to validate sessions, security access, data

identifiers, and fault injection mechanisms safely before the software is integrated into a physical ECU or a Hardware-in-the-Loop (HIL) setup. However, it is important to note the scope limitations of this SIL framework. While the platform excels at verifying diagnostic protocols and application-level safety logic, it remains out-of-scope for evaluating hardware-specific constraints—such as electrical signal latencies, physical microcontroller execution timing, or real peripheral behaviors—which still necessitate downstream HIL validation.

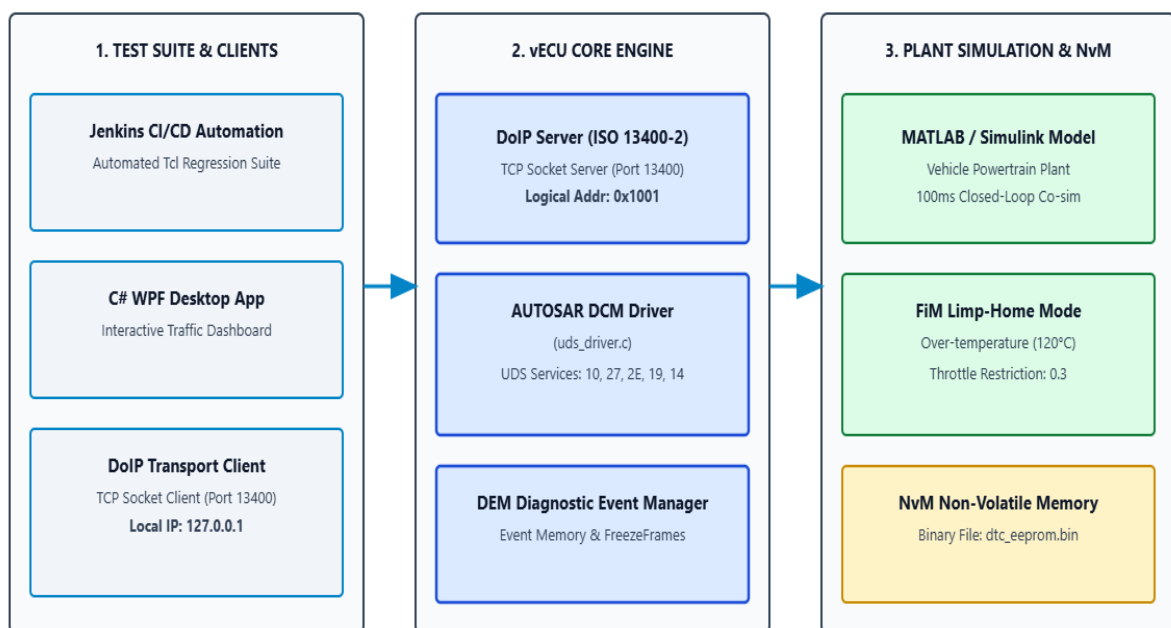


Figure 1.2: vECU Software-in-the-Loop Diagnostic Testing Ecosystem

Software-in-the-loop diagnostic ecosystem linking Jenkins CI/CD, WPF UI, vECU DCM Engine, and Simulink.

1.3 Method

For this interdisciplinary collaborative project to be successful, a structured methodology has been defined that bridges automotive embedded engineering with modern IT automation practices. The first step involves an in-depth study of the ISO 13400 (DoIP) and ISO 14229-1 (UDS) specifications, alongside AUTOSAR diagnostic-service concepts, to establish the theoretical foundation for the vECU. Following the theoretical analysis, the software architecture is designed and implemented by

developing the C-based vECU, a MATLAB/Simulink signal model to represent the engine plant, a WPF Desktop control surface, and a set of Tcl scripts for test automation. The final phase concerns the orchestration of the automated testing environment where a Jenkins CI/CD server is utilized to execute regression test suites whenever source code changes occur. This pipeline compiles the code, runs the vECU, executes the Tcl scripts, and preserves logs and HTML reports as build artifacts to ensure strict and continuous quality assurance.

CHAPTER 2. RESEARCH AND ANALYSIS

2.1 Use Cases for the Simulation Platform

To motivate the development of the framework, several useful operational areas have been identified that define what developers and test engineers can gain by utilizing the simulation software. The first use case addresses interactive development testing, where engineers require the ability to validate modified vECU features manually. The platform accommodates this by allowing developers to connect to the vECU via the Desktop UI, enabling live traffic monitoring and direct interaction with the simulated diagnostic endpoints. The second use case focuses on automated regression. In modern software engineering, manual testing is insufficient for rapid iterations; therefore, the system integrates with a Jenkins pipeline to automatically rerun the complete test suite after any source code changes. The third critical use case is the validation of safety mechanisms through fault injection. Testing severe fault conditions on a physical engine is inherently dangerous, but the virtual simulation circumvents this risk by allowing simulated coolant over-temperature inputs to safely trigger the Diagnostic Trouble Code (DTC) creation process and validate the corresponding safety reactions.

2.2 DoIP and UDS Architectural Design

The communication architecture of the prototype relies on the Diagnostics over Internet Protocol (DoIP), adhering to the ISO 13400 standard, where the vECU exposes its diagnostic interface over TCP port 13400. A crucial mechanism within DoIP is the routing activation sequence, which dictates that an external tester must first request routing activation by transmitting a specific DoIP payload of type **0x0005**. The vECU must process this request and return an activation response of **0x0006** before it permits the transmission of any subsequent diagnostic traffic. Once the network layer is activated, the application layer diagnostic behavior is governed by the Unified Diagnostic Services (UDS) protocol, as specified by ISO 14229-1.

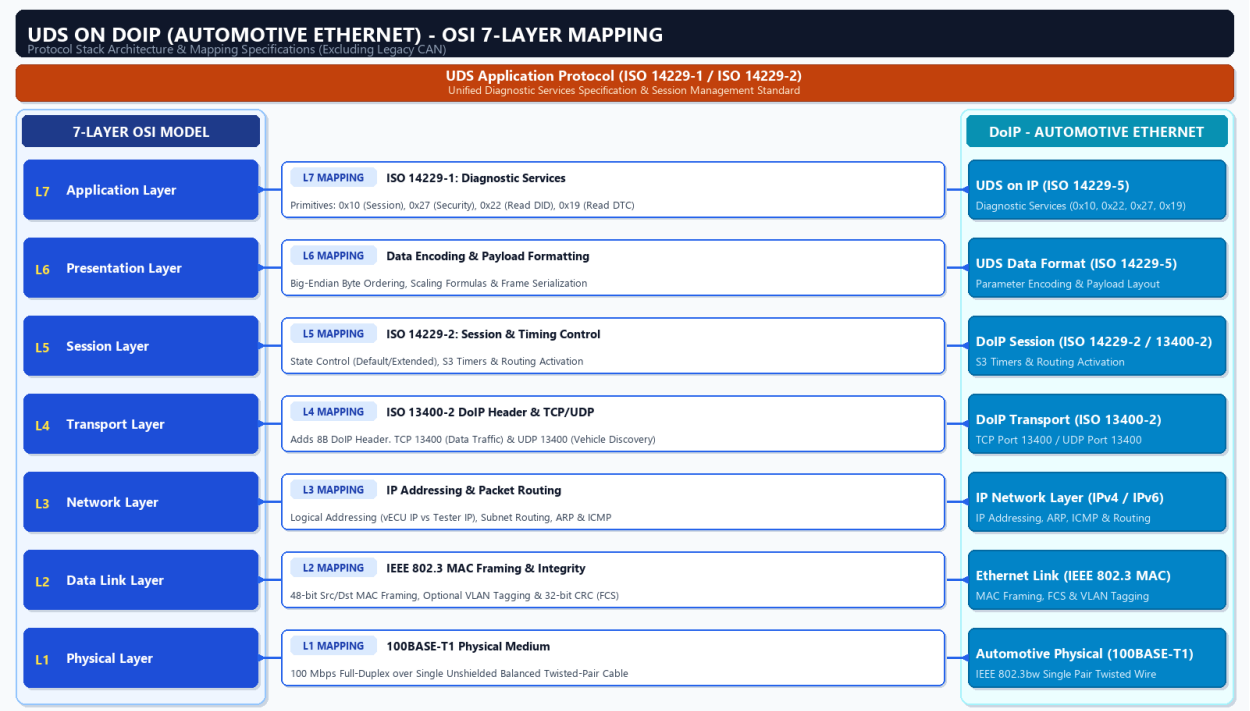


Figure 2.1: 7-Layer OSI Model vs UDS ISO Standards

Mapping of 7-Layer OSI model to UDS on DoIP (ISO 13400-2 / IEEE 802.3).

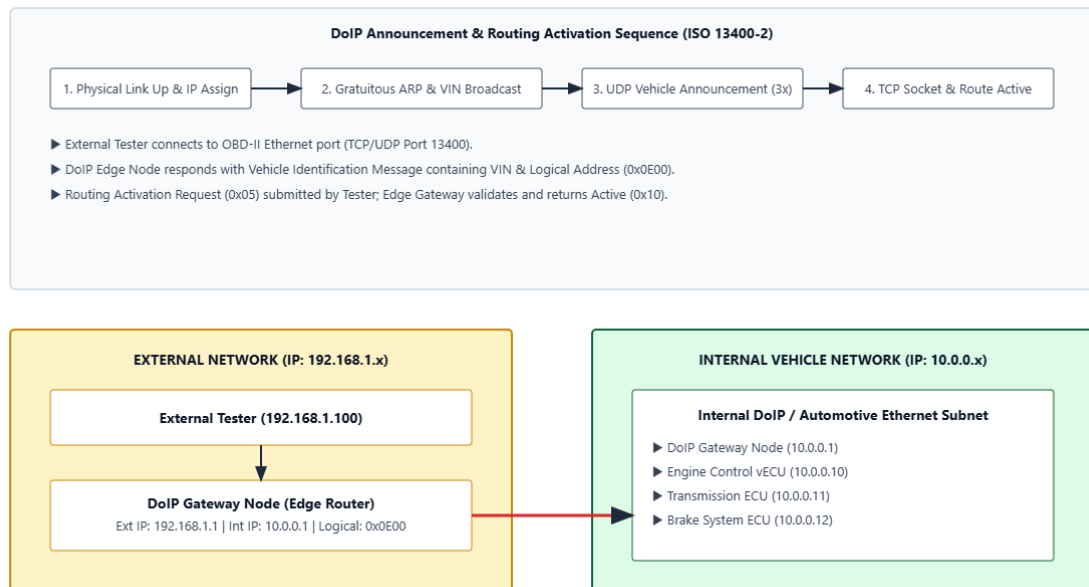


Figure 2.2: DoIP Discovery, Vehicle Identification & Network Topology

UDP broadcast announcement sequence and internal/external IP subnet routing architecture.

The prototype implements a specific subset of these diagnostic services to fulfill its operational requirements. Session management is handled by the **DiagnosticSessionControl (0x10)** service, which allows the tester to shift the vECU from the Default session to the Extended Session using the request **10 03**, returning a positive response of **50 03**. Certain diagnostic operations are protected and require the **SecurityAccess (0x27)** service, which enforces a Seed-Key authentication handshake. The tester requests a seed via **27 01** and subsequently submits a computed key via **27 02**. A successful validation unlocks the system with a **67 02** response, while a failure results in a Negative Response Code, such as **7F 27 35** for an invalid key. Data interaction is facilitated by the **ReadDataByIdentifier (0x22)** service, utilized to read the Vehicle Identification Number (VIN) or to poll simulated signals like vehicle speed, alongside the **WriteDataByIdentifier (0x2E)** service, which is used to write supported data or inject simulated faults into the system. For fault management, the framework relies on the **ReadDTCInformation (0x19)** service to read the status of active faults, and the **ClearDiagnosticInformation (0x14)** service to clear the stored diagnostic memory. Finally, to maintain an active session and prevent the S3 server timeout, the tester must periodically transmit a **TesterPresent (0x3E)** message with the sub-function **80** to keep the session alive while suppressing a positive response.

To provide a comprehensive technical reference for the diagnostic architecture, Table 2.1, Table 2.2, and Table 2.3 summarize the complete set of ISO 14229-1 UDS Service Identifiers (SIDs), Data Identifiers (DIDs), and Negative Response Codes (NRCs) implemented and validated within the vECU diagnostic stack.

Service ID (Hex)	UDS Service Name (ISO 14229-1)	Sub-function / Parameters	Response SID (Hex)	Description & Project Implementation Details
0x10	DiagnosticSessionControl	0x01: Default Session 0x03: Extended Session	0x50	Controls the active diagnostic session state. vECU boots into Default Session (0x01) with security locked. Requesting 10 03 transitions to Extended Session (0x03), enabling SecurityAccess and write services.
0x27	SecurityAccess	0x01: Request Seed 0x02: Send Key	0x67	Executes Seed-Key challenge-response authentication in Extended Session. Sub-function 0x01 requests a 16-bit random seed. Sub-function 0x02 validates key: $\text{Key} = (\text{Seed} \wedge 0x5A5A) + 0x1234$ to grant UNLOCKED state.

Service ID (Hex)	UDS Service Name (ISO 14229-1)	Sub-function / Parameters	Response SID (Hex)	Description & Project Implementation Details
0x22	ReadDataByIdentifier	DID (2 Bytes): 0x0100, 0x0101, 0x0102, 0x01FF, 0xF190	0x62	Reads real-time vehicle parameters (Speed, RPM, Coolant Temp), Simulink co-simulation counter, or static Vehicle Identification Number (VIN = "AUTVECU") from vECU RAM structures.
0x2E	WriteDataByIdentifier	DID (2 Bytes): 0x0100, 0x0101, 0x0102, 0x0105	0x6E	Writes operational parameter values into vECU RAM. Utilized by Simulink plant co-simulation and automated fault injection scripts (injecting overheat temp $\geq 120^{\circ}\text{C}$ / 182°C / 183°C). Requires Extended Session & Unlocked Security.

Service ID (Hex)	UDS Service Name (ISO 14229-1)	Sub-function / Parameters	Response SID (Hex)	Description & Project Implementation Details
0x19	ReadDTCInformation	0x02: Report DTC By Status Mask (0xFF)	0x59	Queries Diagnostic Event Manager (DEM) event memory for active DTCs. Returns active DTC 0x000115 (P0115 Engine Coolant Sensor High) when coolant temp exceeds safety threshold.
0x14	ClearDiagnosticInformation	Memory Group: 0xFFFFFFFF (All DTCs)	0x54	Clears recorded DTC event records from DEM RAM and purges non-volatile storage (dtc_eeeprom.bin). Requires Extended Session & Unlocked Security.
0x3E	TesterPresent	0x00: Zero SubFunction 0x80: Suppress Pos Resp	0x7E / Suppressed	Keep-alive heartbeat message transmitted periodically by test runners to reset the vECU 5-second S3 Server Timeout timer and maintain Extended Session without timeout reversion.

Table 2.1: Summary of Unified Diagnostic Services (UDS SIDs) Implemented in the vECU Platform

Data Identifier (DID Hex)	Parameter Name	Data Type & Byte Length	Access Right	Description & Operational Value Range
0x0100	Vehicle Speed	uint16_t (2 Bytes)	Read / Write	Real-time vehicle velocity parameter in km/h (e.g., 52 km/h -> Response Hex: 62 01 00 00 34).
0x0101	Engine Speed (RPM)	uint16_t (2 Bytes)	Read / Write	Real-time engine rotational speed in RPM (e.g., 2360 RPM -> Hex 09 38; 3556 RPM -> Hex 0D E4).
0x0102	Coolant Temperature	uint8_t (1 Byte)	Read / Write	Engine coolant temperature in °C (Normal: 75°C–89°C; Overheat Fault Injection: $\geq 120^{\circ}\text{C}$ / 182°C / 183°C triggering DTC P0115 and FiM Limp-Home throttle limit 0.3).
0x0105	Coolant Temp Alias	uint8_t (1 Byte)	Read / Write	Secondary DID alias mapping for engine coolant temperature parameter input.
0x01FF	Simulink Update Counter	uint16_t (2 Bytes)	Read Only	Monotonically increasing step counter verifying active Simulink co-simulation data publication.

Data Identifier (DID Hex)	Parameter Name	Data Type & Byte Length	Access Right	Description & Operational Value Range
0xF190	Vehicle Identification Number (VIN)	ASCII String (7 Bytes)	Read Only	Static ECU hardware identification string ("AUTVECU" -> Hex: 41 55 54 56 45 43 55).

Table 2.2: Supported Diagnostic Data Identifiers (DIDs) in the vECU Diagnostic Stack

NRC Code (Hex)	ISO 14229-1 Standard Name	Trigger Condition & vECU Stack Handling
0x11	serviceNotSupported	Transmitted Service ID (SID) is unknown or not supported by the vECU DCM dispatcher.
0x12	subFunctionNotSupported	Requested sub-function code is invalid or unsupported for the specified service (e.g., SID 0x10 sub-function other than 0x01 or 0x03).
0x13	incorrectMessageLengthOrInvalidFormat	Total byte length of the incoming UDS payload is less than the required minimum parameter length.
0x31	requestOutOfRange	Requested Data Identifier (DID) is unrecognized or written parameter value exceeds operational bounds.

NRC Code (Hex)	ISO 14229-1 Standard Name	Trigger Condition & vECU Stack Handling
0x33	securityAccessDenied	Privileged diagnostic operation (Write 0x2E or Clear 0x14) is requested while Security state is LOCKED.
0x35	invalidKey	Submitted key in 27 02 does not match expected algorithm formula: $\text{Key} = (\text{Seed} \wedge 0x5A5A) + 0x1234$.
0x7E	subFunctionNotSupportedInActiveSession	Sub-function is prohibited in the current active diagnostic session.
0x7F	serviceNotSupportedInActiveSession	Privileged service (SecurityAccess 0x27 or Write 0x2E) requested in Default Session (0x01) instead of Extended Session (0x03).

Table 2.3: Implemented Negative Response Codes (NRCs) Matrix in vECU Diagnostic Stack

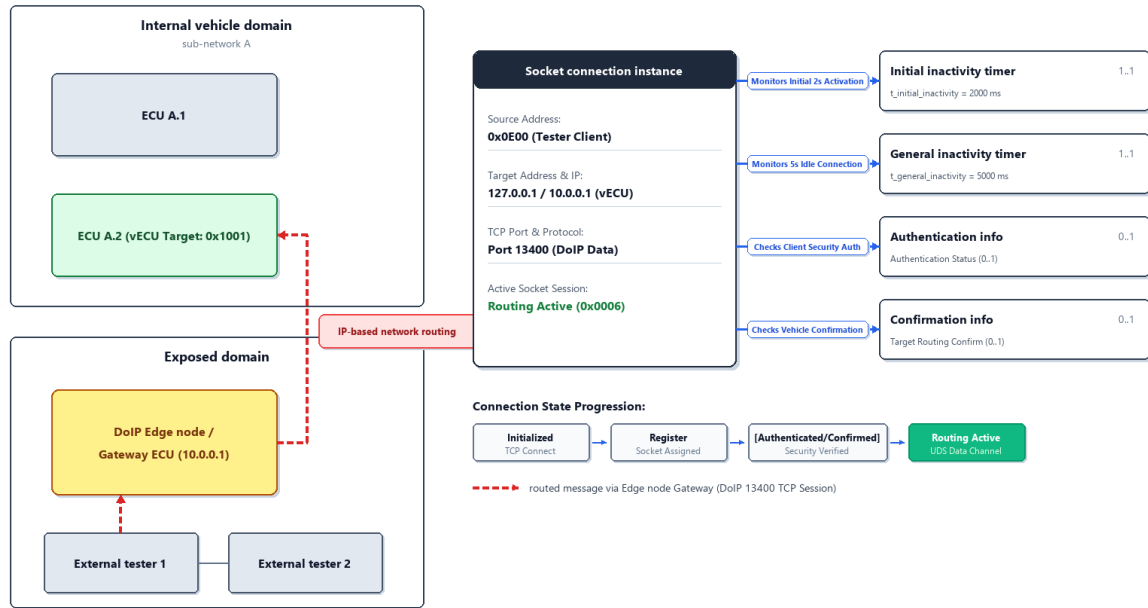


Figure 2.3: DoIP Gateway Topology & Socket Connection Instance Model
Gateway ECU routing payload over IP-based network to internal vehicle domain ECUs.

2.3 Overview of AUTOSAR Diagnostic Stack Architecture

To establish a standardized software topology, modern automotive electronic control units are heavily structured around the AUTOSAR (AUTomotive Open System Architecture) Classic Platform. Within the AUTOSAR Basic Software (BSW) layer, diagnostic communication and fault management are primarily governed by four core modules operating in the Service Layer:

- Diagnostic Communication Manager (DCM):** Positioned between the PDU Router (PduR) and the Runtime Environment (RTE), the DCM handles the OSI layers 5 to 7 of diagnostic protocols. It consists of three sub-layers: the Diagnostic Session Layer (DSL) for session/security state management, the Diagnostic Service Dispatcher (DSD) for parsing incoming UDS requests, and the Diagnostic Service Processing (DSP) for executing service-specific handler routines.

- **Diagnostic Event Manager (DEM):** Responsible for processing, filtering, and persisting diagnostic events reported by onboard monitors. The DEM manages the DTC status byte, handles freeze frame storage, and interacts with non-volatile memory.
- **Non-Volatile Memory Manager (NvM):** Provides a unified interface to persist critical diagnostic information (such as active DTCs and permanent fault records) into physical EEPROM or flash storage across power cycles.
- **Function Inhibition Manager (FiM):** Evaluates event status from the DEM and evaluates permission states for application software components. If a safety-critical fault occurs, the FiM dynamically restricts or inhibits specific vehicle functions, such as limiting throttle intervention during Limp Home mode.

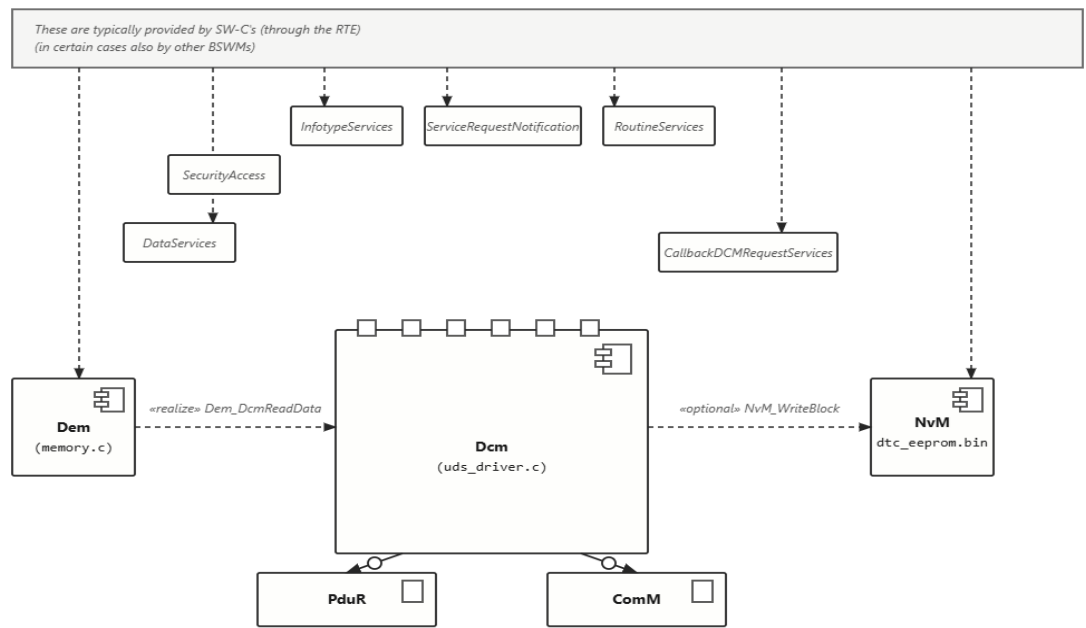


Figure 2.4: Dependencies to other modules (AUTOSAR BSW Architecture)
AUTOSAR Classic BSW DCM module dependencies, RTE software component interfaces, Dem event memory, and NvM persistence.

In this research, while a full commercial AUTOSAR stack introduces heavy configuration overheads, the developed C-based vECU natively models these exact AUTOSAR abstractions (**uds_driver** as DCM, **memory** as DEM/NvM/FiM), bridging

high-level automotive software standards with lightweight Software-in-the-Loop simulation.

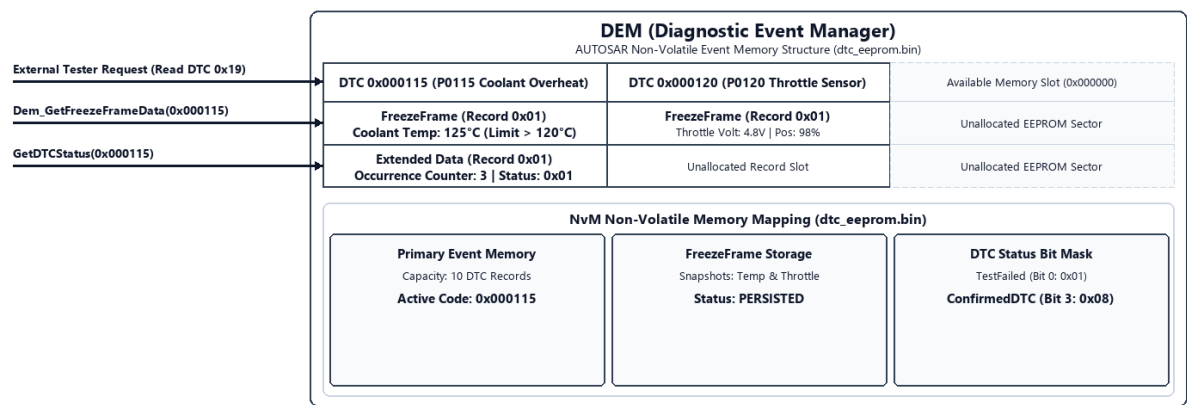


Figure 2.5: DEM Event Memory & FreezeFrame Record Structure

Interactive request/response data flow between DCM and DEM Event Memory for DTC 0x000115 (P0115 Coolant Overheat).

CHAPTER 3. PROTOTYPE DEVELOPMENT

3.1 Overall Software-in-the-Loop Architecture

To address the limitations associated with physical testbenches, the collaborative research framework implements a closed-loop Software-in-the-Loop (SIL) environment partitioned into three distinct, highly synchronized layers. The simulation layer, powered by MATLAB/Simulink, continuously computes engine dynamics and environmental physics, streaming operational parameters such as vehicle speed, engine RPM, and coolant temperature at a 10 Hz frequency. The vECU core, developed as a native Windows C-executable within a structured directory workspace, simulates the internal behavior of an AUTOSAR-compliant Service Layer, integrating diagnostic communication (DCM), event management (DEM), non-volatile memory (NvM), and function inhibition (FiM) modules. Finally, the test and CI/CD layer, orchestrated by a Jenkins server alongside a C# WPF Desktop application and Tcl script runners, acts as an external tester that evaluates the virtual black box over a DoIP network and automatically generates comprehensive HTML reports.

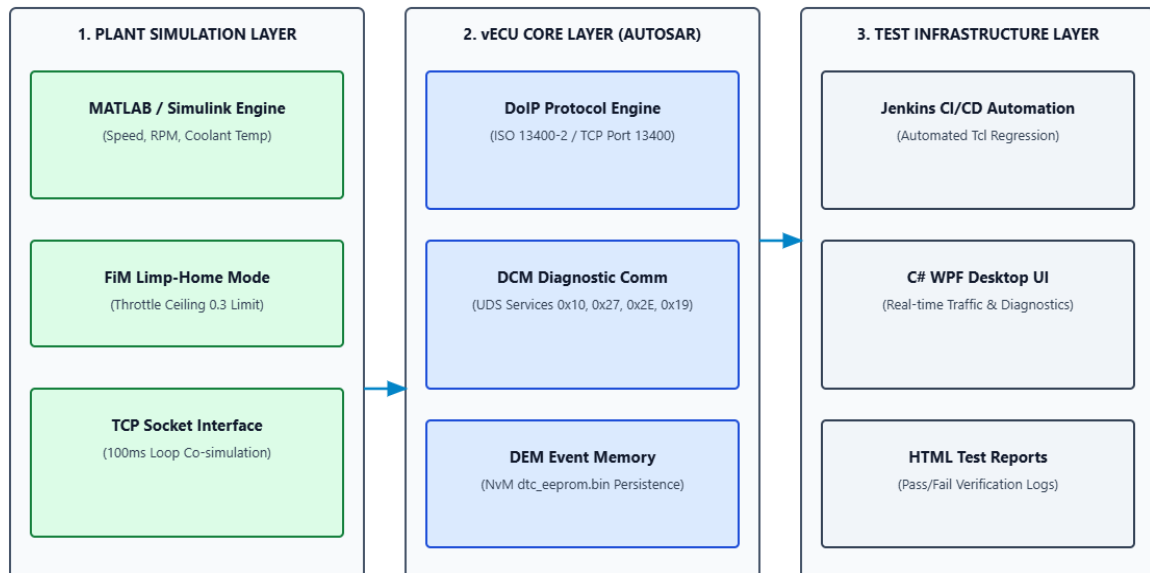


Figure 3.1: Overall SIL Platform Technical Architecture

Software-in-the-Loop closed-loop platform showing Simulink, vECU C Core, and Test Infrastructure.

3.2 Session Management and Security Access State Machine

Modern vehicle networks rely heavily on Ethernet-based transport protocols, leading the prototype to incorporate a custom Diagnostics over Internet Protocol (DoIP) server implemented in **doip_server.c**. The server binds exclusively to the local loopback interface (**127.0.0.1**) on port **13400**, utilizing a non-blocking **select()** loop to manage up to five concurrent client connections efficiently without multi-threading overhead. Before any Unified Diagnostic Services (UDS) payloads are transmitted, external test tools must complete a DoIP routing activation handshake by sending a routing activation request (**0x0005**), prompting the vECU to validate the tester's logical address and return a successful activation response (**0x0006**).

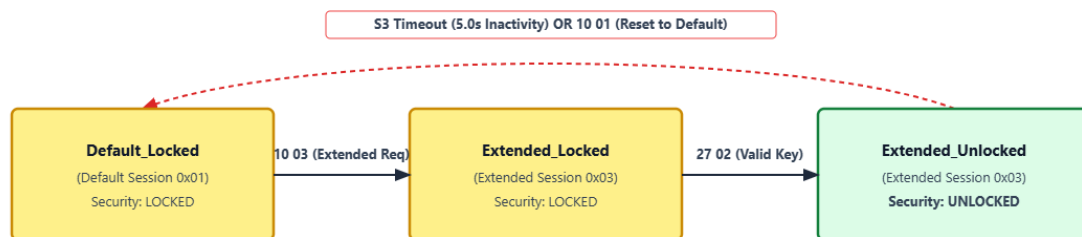


Figure 3.2: Session and Security State Machine

State transitions between Default/Extended Sessions, Seed-Key authentication, and S3 Timeout.

Once the network layer is established, the UDS driver (**uds_driver.c**) enforces the diagnostic session and security state machine. The system boots into the default session (**UDS_SESSION_DEFAULT, 0x01**) with security strictly locked (**g_isSecurityUnlocked = 0**). Transitioning to the extended session (**UDS_SESSION_EXTENDED, 0x03**) via service **10 03** is a mandatory prerequisite for executing privileged operations, such as data writing (**0x2E**) for fault injection. To unlock the system, a mathematical challenge-response protocol is executed via the **SecurityAccess (0x27)** service. The tester requests a seed (**27 01**), and the vECU generates a 16-bit pseudo-random challenge using a time-seeded standard library

rand() function, explicitly avoiding zero-seed generations. The tester then calculates the expected key using the predetermined algorithm $\text{Key} = (\text{Seed} \oplus 0x5A5A) + 0x1234$ and submits it via **27 02**. A successful match unlocks the security state, whereas a mismatch returns a Negative Response Code (NRC) **0x35** for invalid keys. To protect against unauthorized session hijacking, the vECU enforces an S3 Server Timeout mechanism through the **select()** loop; if no diagnostic traffic is registered for five seconds, the system autonomously reverts to the default session and locks security, a timeout that test runners actively suppress by transmitting periodic **TesterPresent (3E 80)** keep-alive frames.

3.3 Simulink Co-Simulation and Limp Home Safety Logic

The framework supports real-time dynamic parameter synchronization through a closed-loop SIL methodology. The MATLAB/Simulink plant model utilizes a specialized **DoIP_Sender_UDS** MATLAB function block to feed operational variables continuously to the vECU via UDS write requests (**0x2E**), updating the **VehicleState_t** RAM structures defined in **memory.h**. To ensure robust long-run stability, the Simulink script incorporates an automated pause-recovery mechanism using the **etime** function; if simulation pauses exceed four seconds—which would otherwise trigger an S3 timeout on the vECU—the script automatically re-initiates the DoIP routing activation and the complete UDS Seed-Key handshake sequence to re-establish the diagnostic channel seamlessly.

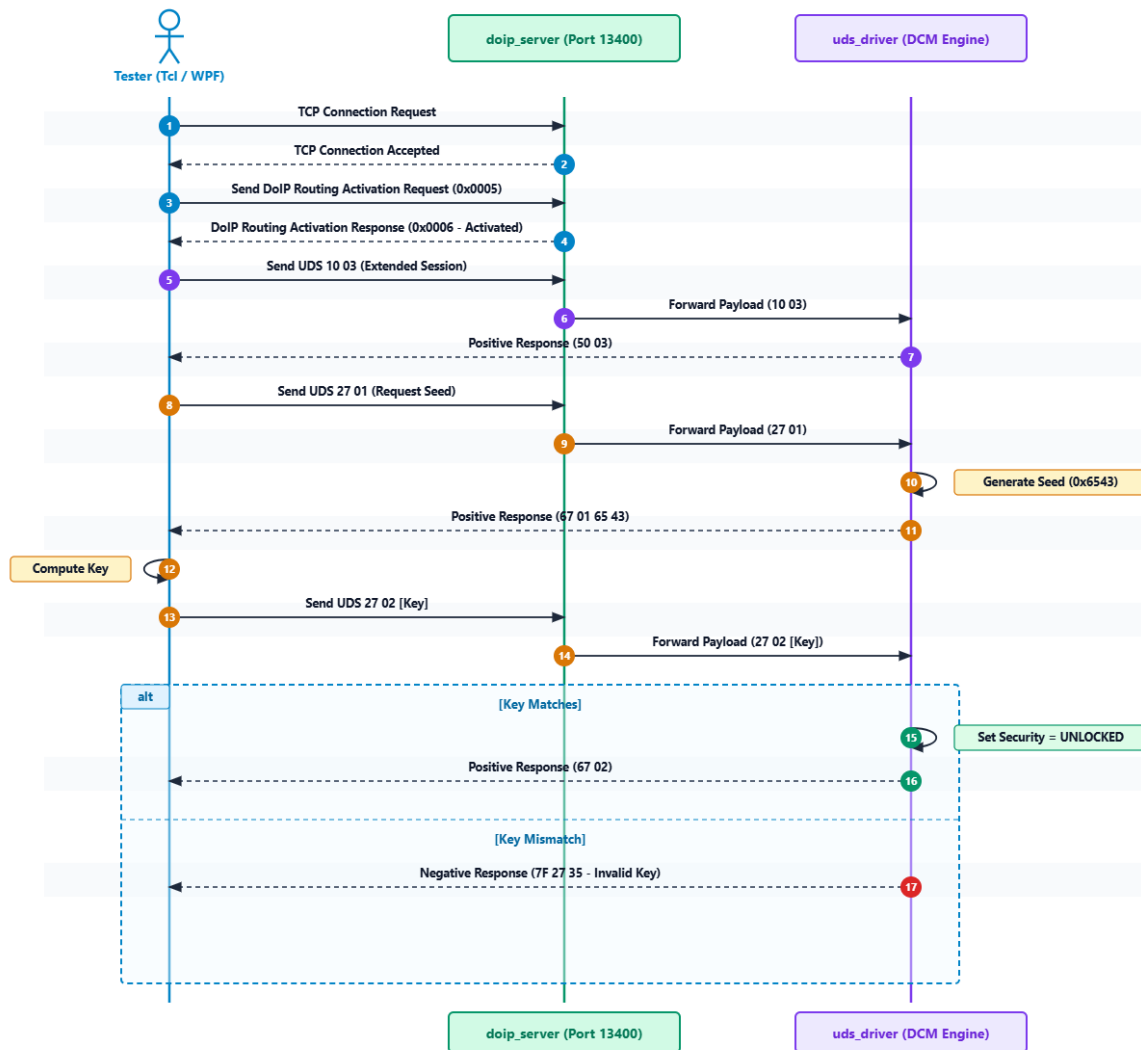


Figure 3.3: UDS Diagnostic & Security Access Execution Sequence

Detailed interaction sequence between Tester, DoIP Server, and DCM Engine for Session & Security Access.

Within the vECU's **memory.c**, the Diagnostic Event Manager (DEM) monitors the incoming coolant temperature to simulate safety-critical boundary conditions. When the temperature equals or exceeds 120°C, the vECU triggers diagnostic trouble code (DTC) **0x000115**, setting its status mask to active (**0x01**) and invoking **SaveDTCsToDisk()** to persist the fault state into the non-volatile memory (NvM) binary file **dtc_eeeprom.bin** via standard file I/O operations. To prevent erratic throttle chattering caused by minor sensor fluctuations, the algorithm introduces a hysteresis recovery filter, ensuring the fault remains latched until the coolant temperature drops back below a safe threshold of

$\leq 100^{\circ}\text{C}$. When the Simulink plant polls the vECU via **19 02 FF** and detects the active **P0115** fault (processed through a 10-cycle debounce filter to eliminate transient glitches), its Function Inhibition Manager (FiM) logic actively intervenes by limiting the throttle actuation position to a maximum safety ceiling of **0.3**, thereby verifying the platform's capability to execute automated Limp Home safety reactions.

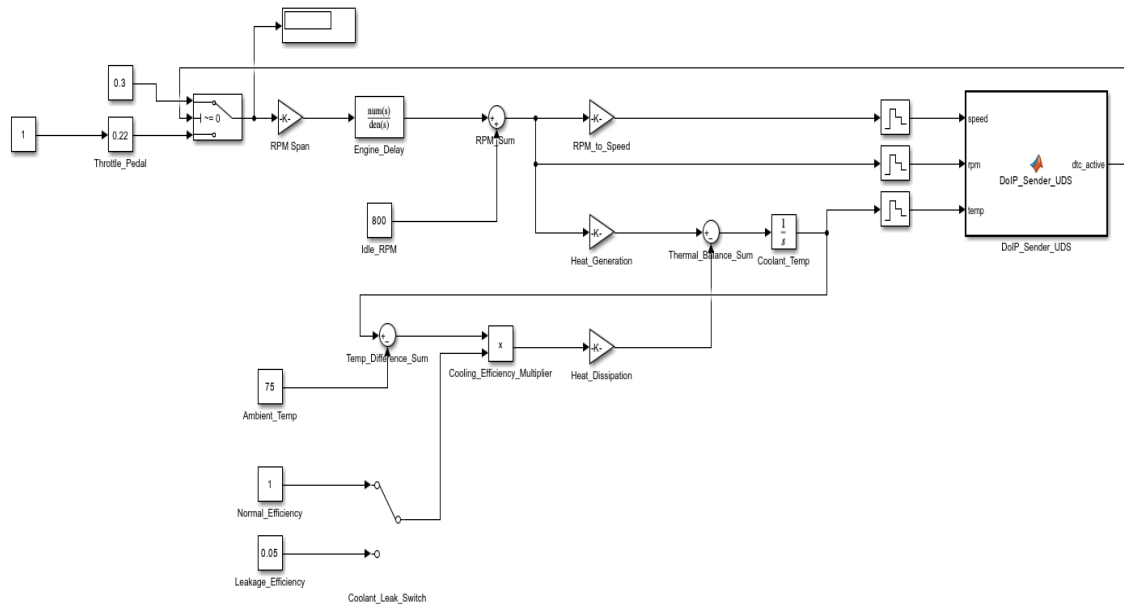


Figure 3.4: MATLAB/Simulink Closed-Loop Powertrain Plant Model & FiM Limp-Home Logic

Simulink block diagram illustrating 100ms fixed-step co-simulation, engine thermal dynamics, and FiM safety restriction loop limiting maximum throttle to 30% upon DTC P0115 activation.

3.4 Automated Testing Framework and Jenkins CI/CD Pipeline

To transition from manual verification to continuous quality assurance, the research integrates an automated testing framework driven by a C# WPF Desktop application and a Tcl command-line script runner. Test sequences defined in JSON formats are dynamically parsed and converted into executable Tcl scripts via a dedicated generator service. To prevent test execution lockups or hanging background processes, a robust "Stop Test" cancellation mechanism is engineered into **tclsh_wrapper.c**, ensuring that

terminating an active test token immediately destroys the associated **telsh** process tree using native OS calls (**TerminateProcess**), logging the execution state as safely stopped rather than yielding false pass or fail outcomes.

The DevOps integration is realized through a declarative **Jenkinsfile** that orchestrates a six-stage continuous integration pipeline. The pipeline sequentially executes agent validation, solution restoration and compilation in Release configuration, unit testing of core libraries (**AutVecu.Cores**), background vECU deployment via batch scripts (**start_vecu.bat**) with port 13400 availability checks, UDS protocol regression test execution (**run_tests.bat**), and a mandatory post-action cleanup block (**stop_vecu.bat**) that forcefully terminates **vECU.exe** to release network sockets before archiving comprehensive HTML/JSON test reports and execution logs as build artifacts.

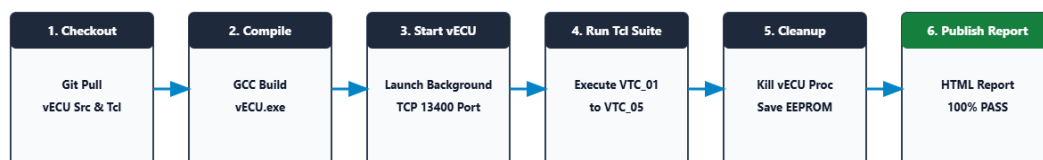


Figure 3.5: Jenkins Automated CI/CD Regression Pipeline

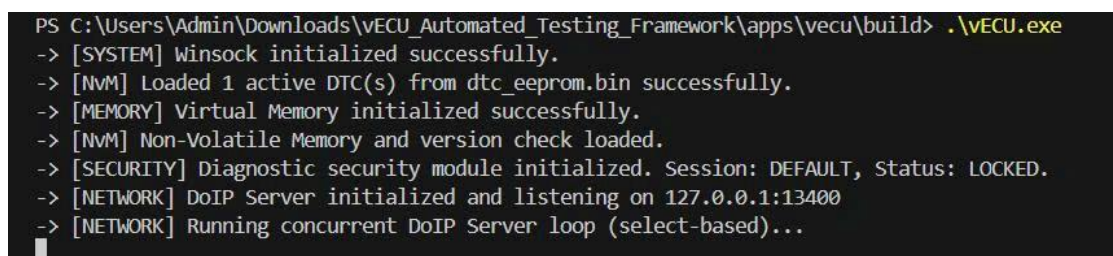
Six-stage automated pipeline: Checkout -> GCC Compile -> Start vECU -> Run Tcl Suite -> Kill Process -> Publish Report.

CHAPTER 4. TESTING, RESULTS, AND EVALUATION

4.1 Verification and Test Execution

To validate the operational reliability of the proposed Software-in-the-Loop (SIL) framework, a series of automated and interactive test suites were executed. The testing methodology evaluates the vECU's capability to process standard Unified Diagnostic Services (UDS) over the custom Diagnostics over Internet Protocol (DoIP) transport layer. Test scenarios are systematically structured, ranging from basic session transitions (**10 03**) and security access handshakes (**27 01**, **27 02**) to advanced boundary condition evaluations such as fault injection and safety inhibition. During interactive execution via the C# WPF Desktop application, operators can monitor live message traffic, track parameter updates from the MATLAB/Simulink plant model, and verify that invalid service requests or locked security states correctly trigger appropriate Negative Response Codes (NRCs), such as **0x35** for invalid keys or **0x7F** for session errors.

During initialization, the vECU process (vECU.exe) initializes Winsock networking, loads active DTC records from `dtc_eeprom.bin`, allocates virtual memory, and configures the diagnostic security module into its default locked state (DEFAULT / LOCKED). The DoIP Server subsequently binds to TCP port 13400 on IP 127.0.0.1 and runs an asynchronous `select()` event loop.

A terminal window with a black background and white text. The prompt is 'PS C:\Users\Admin\Downloads\VECU_Automated_Testing_Framework\apps\vecu\build>'. The command entered is './vecu.exe'. The output consists of several status messages: '[SYSTEM] winsock initialized successfully.', '[NVM] Loaded 1 active DTC(s) from dtc_eeprom.bin successfully.', '[MEMORY] Virtual Memory initialized successfully.', '[NVM] Non-Volatile Memory and version check loaded.', '[SECURITY] Diagnostic security module initialized. Session: DEFAULT, Status: LOCKED.', '[NETWORK] DoIP Server initialized and listening on 127.0.0.1:13400', and '[NETWORK] Running concurrent DoIP Server loop (select-based)...'.

```
PS C:\Users\Admin\Downloads\VECU_Automated_Testing_Framework\apps\vecu\build> ./vecu.exe
-> [SYSTEM] winsock initialized successfully.
-> [NVM] Loaded 1 active DTC(s) from dtc_eeprom.bin successfully.
-> [MEMORY] Virtual Memory initialized successfully.
-> [NVM] Non-Volatile Memory and version check loaded.
-> [SECURITY] Diagnostic security module initialized. Session: DEFAULT, Status: LOCKED.
-> [NETWORK] DoIP Server initialized and listening on 127.0.0.1:13400
-> [NETWORK] Running concurrent DoIP Server loop (select-based)...
```

Figure 4.1: vECU C-Core Execution and DoIP Server Startup Terminal Log

The DoIP Server manages client TCP connections dynamically. When a test client or Jenkins regression runner connects, a slot is assigned and handled independently, followed by clean resource deallocation upon disconnect.

```
-> [NETWORK] Accepted new connection at slot 1 from 127.0.0.1:52728
-> [NETWORK] client disconnected from slot 1.
```

Figure 4.2: TCP Socket Connection Acceptance and Disconnection Log

In standard operation, the Simulink powertrain model (UWAF_T_Blazer_P4_4WD_Opt) receives driver input via the Throttle_Pedal block (Slider Gain = 0.530). As highlighted in the annotated diagram, the Coolant_Leak_Switch is connected to Normal_Efficiency (1.0), representing the Normal Operating State where thermal dissipation operates at 100% capacity, maintaining stable coolant temperature within normal bounds (75°C - 89°C).

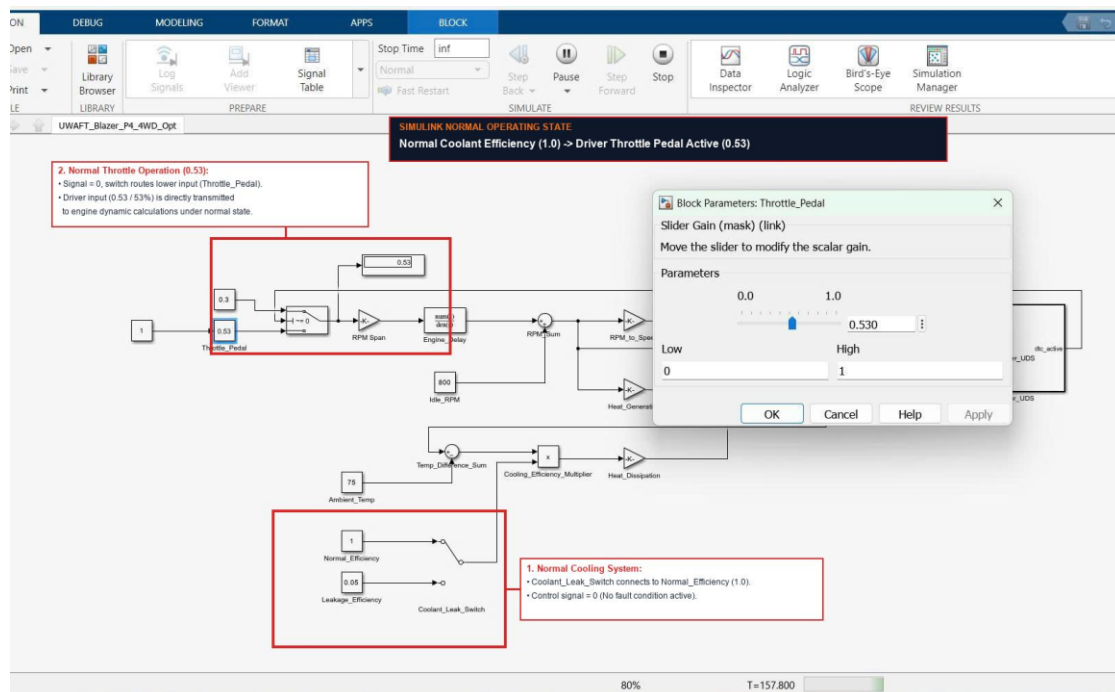


Figure 4.3: MATLAB/Simulink Powertrain Model - Normal Operating State
(Coolant_Leak_Switch = 1.0)

To test the Diagnostic Event Manager (DEM) and Function Inhibition Manager (FiM), a coolant leak fault injection scenario is modeled by switching Coolant_Leak_Switch to a degraded efficiency factor (Leakage_Efficiency = 0.05), inducing engine thermal escalation.

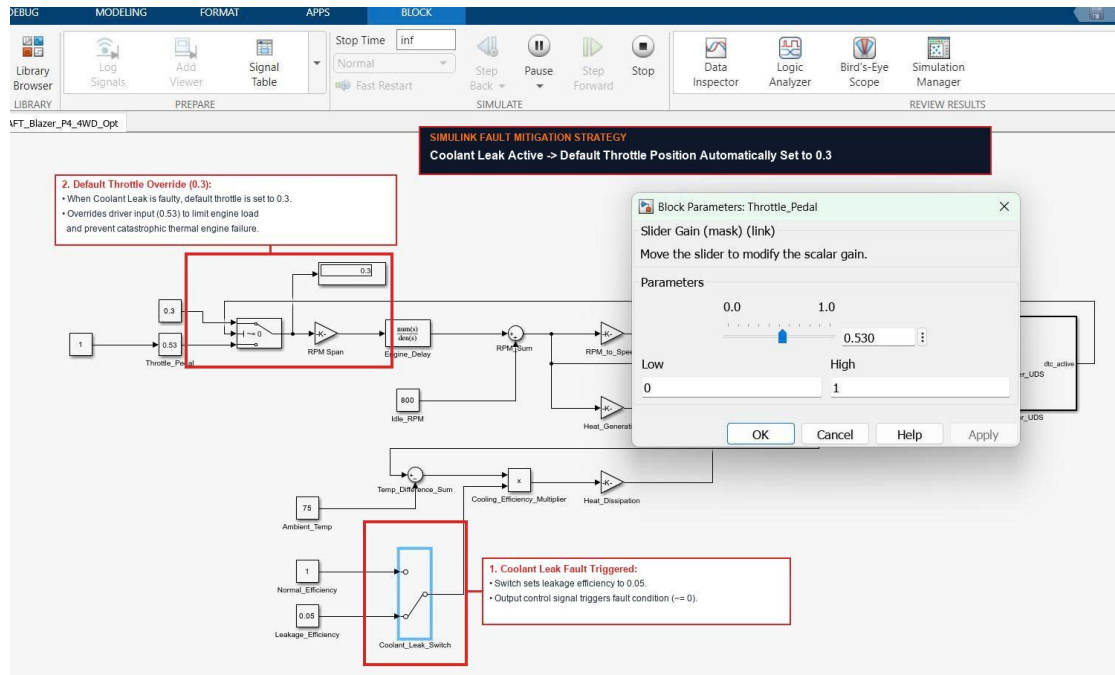


Figure 4.4: MATLAB/Simulink Powertrain Model - Coolant Leakage Fault Injection Setup (Leakage_Efficiency = 0.05, Throttle Limit = 0.3)

During co-simulation, physical variables calculated by Simulink (such as RPM = 3556 and Coolant Temp = 89°C) are updated continuously in the vECU RAM using UDS 0x2E WriteDataByIdentifier requests.


```

... PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

| +-- Service Name      : 0x2E (WriteDataByIdentifier)
| +-- Target DID/Sub    : 0x0101
|
| 3. APPLICATION LAYER (ASW & Virtual Memory)
| +-- Operation         : Writing Engine RPM
| +-- Result Value      : New RPM = 3556 RPM
|
| 4. RESPONSE GENERATION (DoIP Encapsulation)
| +-- Raw Response Hex  : 6E 01 01
| +-- Sent DoIP Frame   : 15 bytes (swapped src/target address)
|
+-----+

+-----+
| [DoIP FLOW] DATA PROCESSING FLOW VISUALIZATION
+-----+

| 1. NETWORK LAYER (DoIP Packet Received)
| +-- Protocol Version : 0x02
| +-- Payload Type     : 0x8001 (Diagnostic Message)
| +-- Source Address    : 0x0E00 (Test Client)
| +-- Target Address    : 0x0E80 (vECU Target)
|
| 2. DIAGNOSTIC LAYER (UDS Extracted)
| +-- Raw Request Hex   : 2E 01 02 59
| +-- Service Name      : 0x2E (WriteDataByIdentifier)
| +-- Target DID/Sub    : 0x0102
|
| 3. APPLICATION LAYER (ASW & Virtual Memory)
| +-- Operation         : Writing Coolant Temp (Fault Injection)
| +-- Result Value      : New Temp = 89 C
|
| 4. RESPONSE GENERATION (DoIP Encapsulation)
| +-- Raw Response Hex  : 6E 01 02
| +-- Sent DoIP Frame   : 15 bytes (swapped src/target address)
|
+-----+

```

Figure 4.5: Dynamic Co-Simulation Data Transfer Log from Simulink to vECU RAM via UDS 0x2E

Verification of Unified Diagnostic Services (UDS) over DoIP transport includes validating data read primitives (0x22 ReadDataByIdentifier). The vECU extracts the target Data Identifier (DID) from incoming DoIP payload frames (Payload Type 0x8001) and returns positive responses with raw hexadecimal byte arrays:

- Vehicle Speed (DID 0x0100): Returned value is 52 km/h (UDS Response Hex: 62 01 00 00 34).
- Engine RPM (DID 0x0101): Returned value is 2360 RPM (UDS Response Hex: 62 01 01 09 38).
- Coolant Temperature (DID 0x0102): Returned value is 89°C (UDS Response Hex: 62 01 02 59).

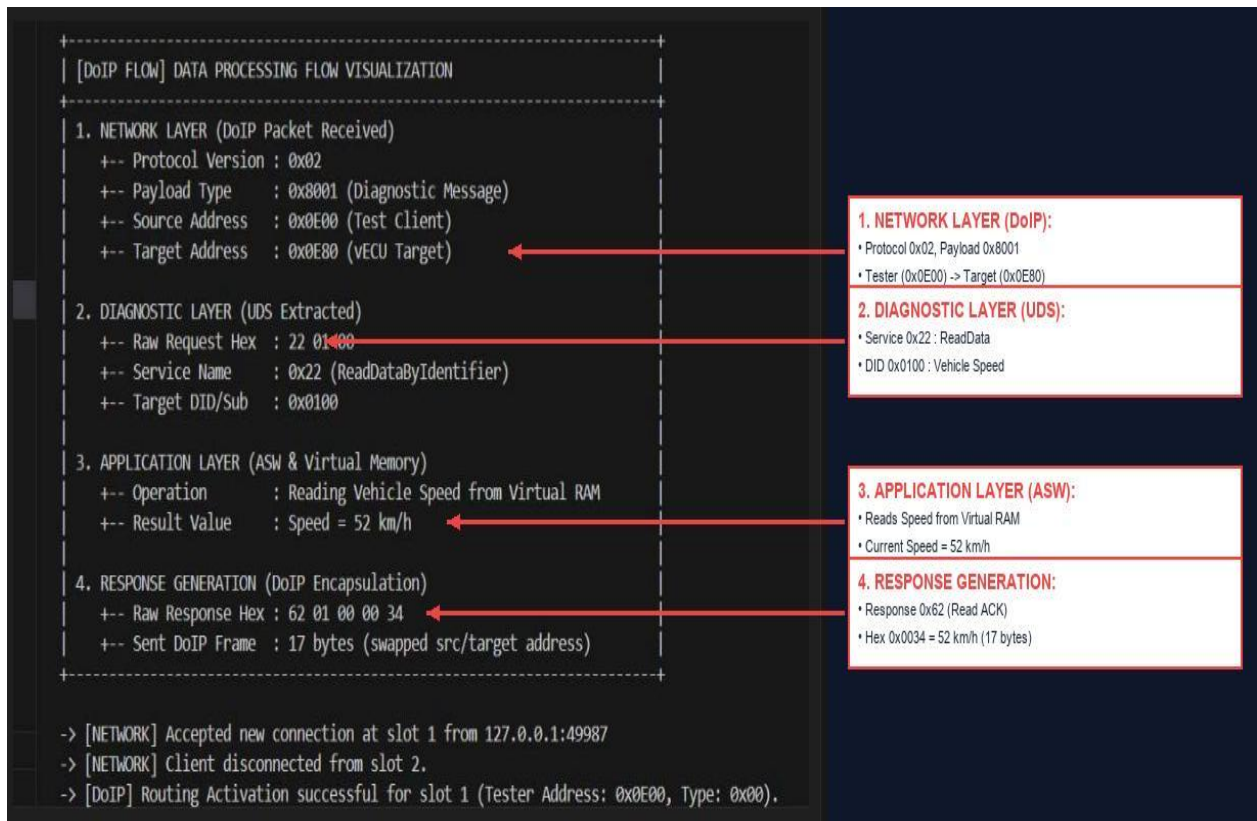


Figure 4.6: DoIP/UDS Data Processing Flow Log - Read Vehicle Speed (DID 0x0100)

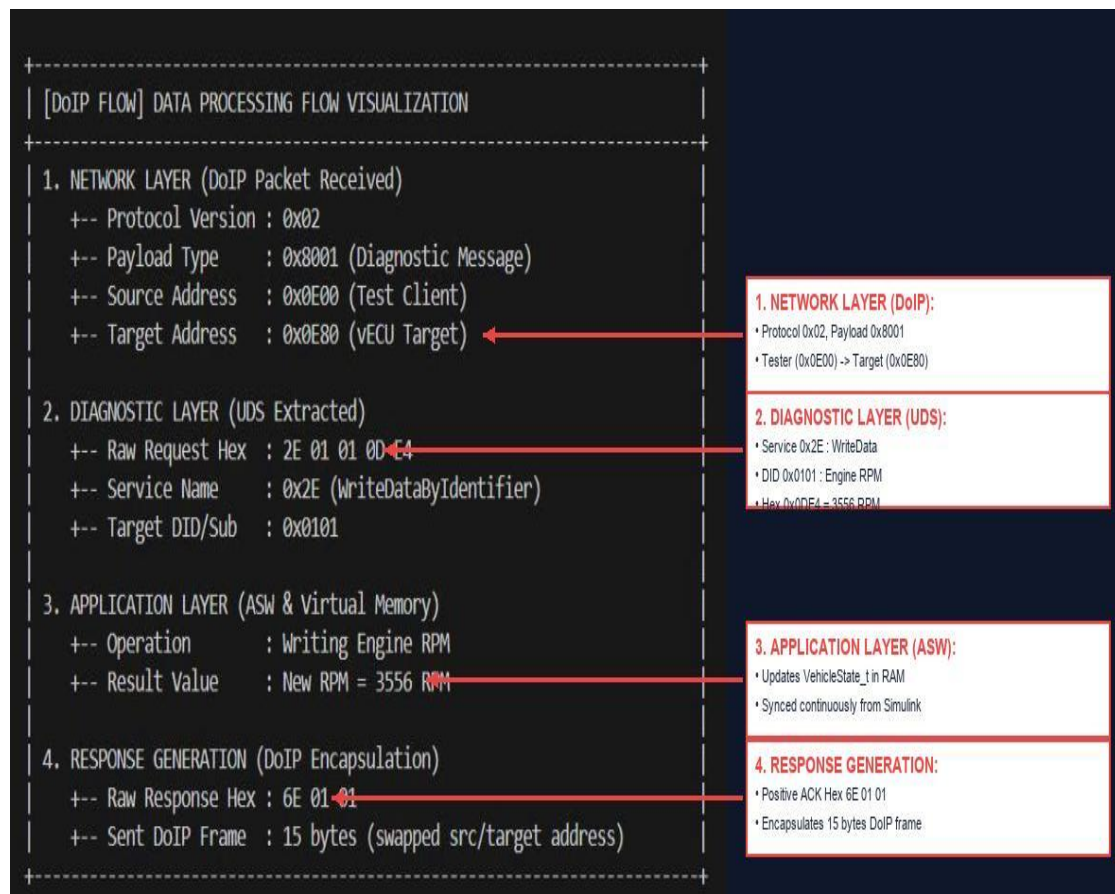


Figure 4.7: DoIP/UDS Data Processing Flow Log - Read Engine RPM (DID 0x0101)

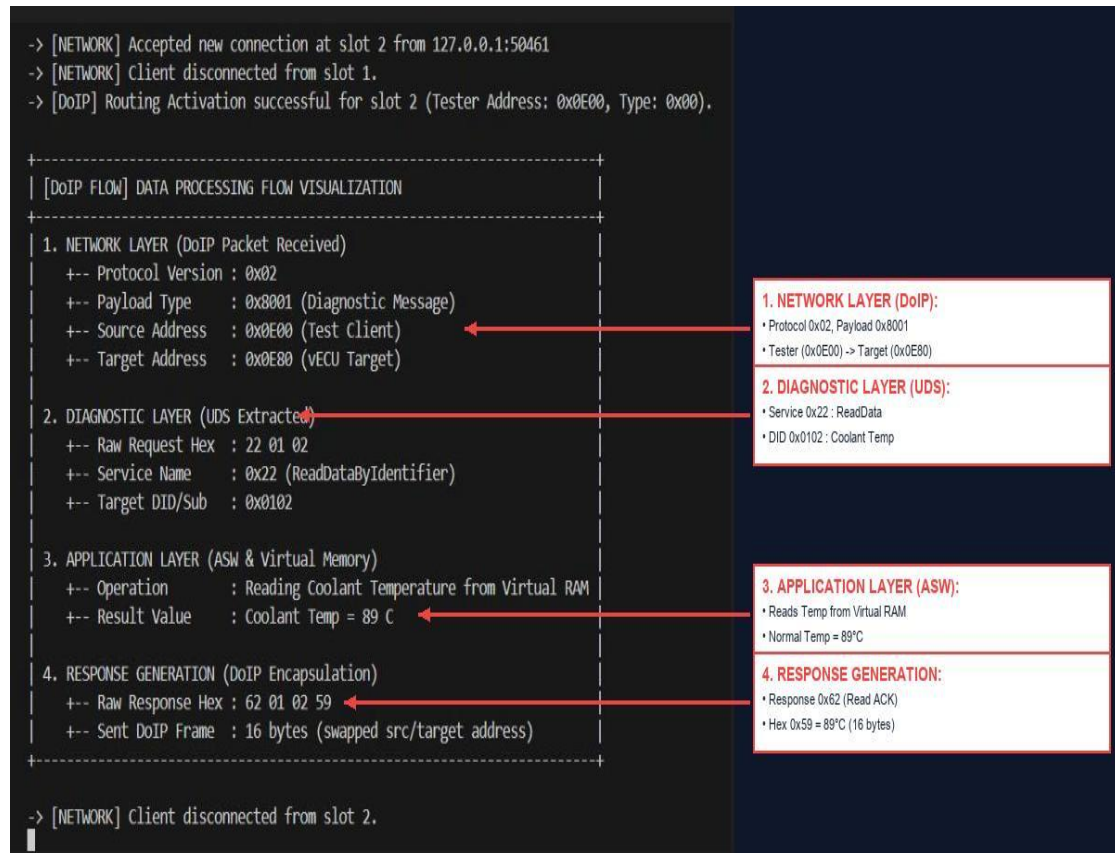


Figure 4.8: DoIP/UDS Data Processing Flow Log - Read Normal Coolant Temperature (DID 0x0102 = 89°C)

Fault injection testing evaluates the vECU's DEM module by transmitting UDS 0x2E requests with elevated coolant temperatures (182°C and 183°C). Upon receiving out-of-bounds parameters, the vECU triggers diagnostic trouble code DTC P0115 (DTC P0115 Triggered!) and persists the error status into dtc_eeprom.bin.

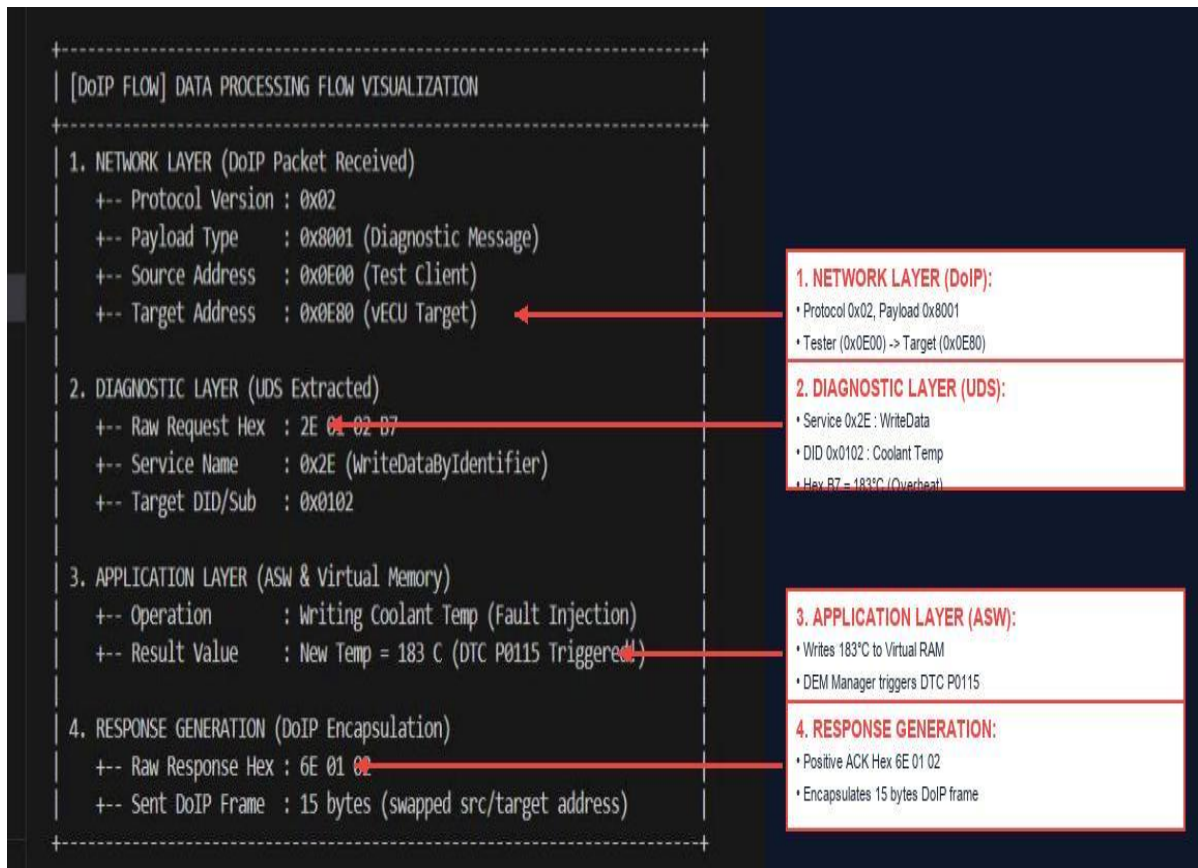


Figure 4.9: DoIP/UDS Fault Injection Log - Write Overheat Temperature (183°C) and DTC P0115 Recording

Interactive testing was conducted via the custom C# WPF Desktop Dashboard (AUT vECU Test Automation). When running standard diagnostic test sequences under normal conditions, the application confirms a 100% pass rate (4/4 Passed) with real-time DoIP message logs and positive ACKs (0x0006, 0x8002).

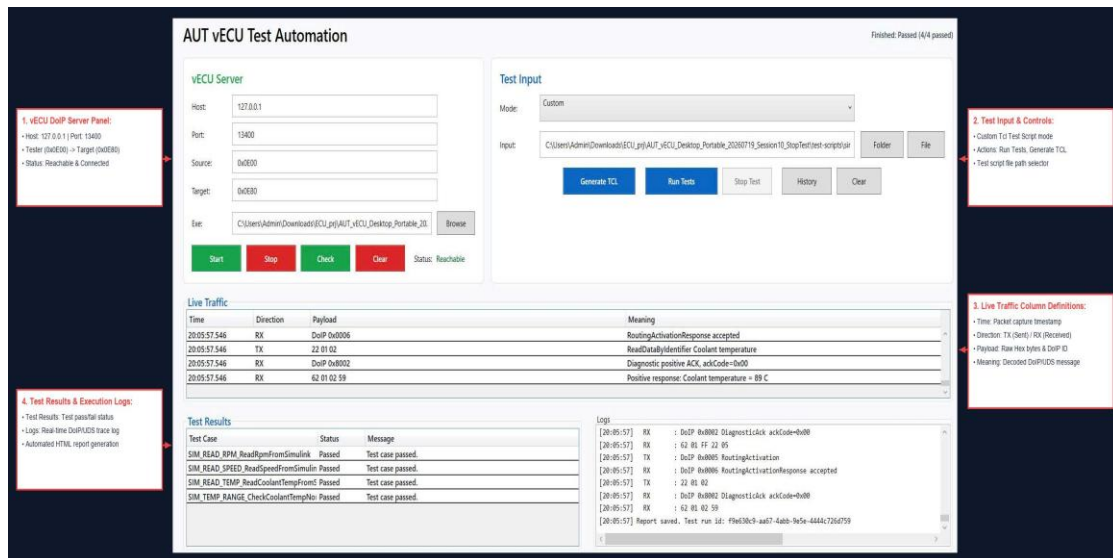


Figure 4.10: C# WPF Test Automation Dashboard - Successful Test Suite Execution (4/4 Passed)

Conversely, when evaluating boundary check test cases with abnormal thermal parameters (e.g., Coolant Temp = 195°C), the test automation engine detects assertion violations (Actual response did not satisfy assertion) and highlights the failed test case (3/4 Passed).

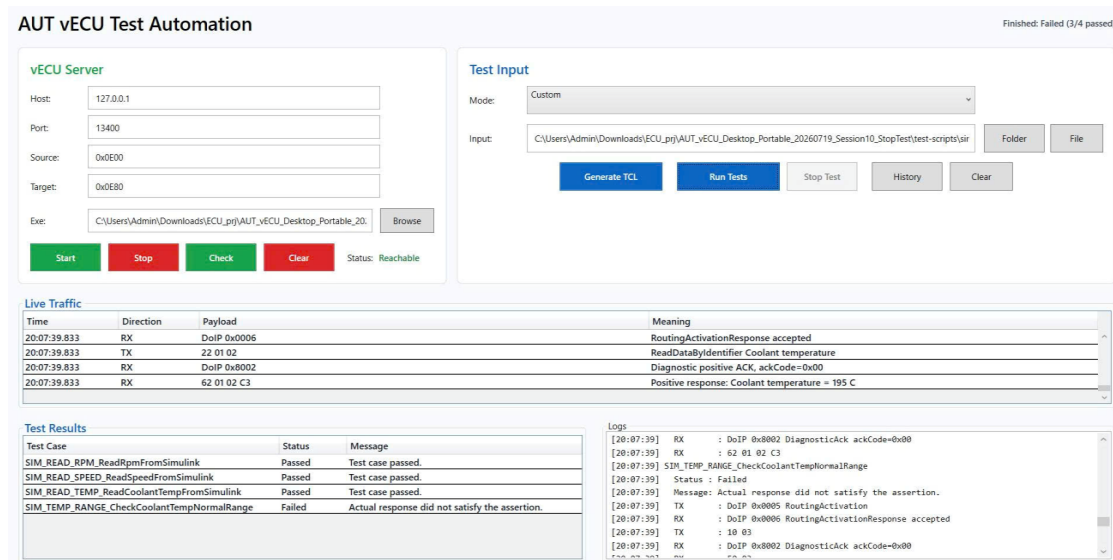


Figure 4.11: C# WPF Test Automation Dashboard - Assertion Failure Detection on Overheat Condition

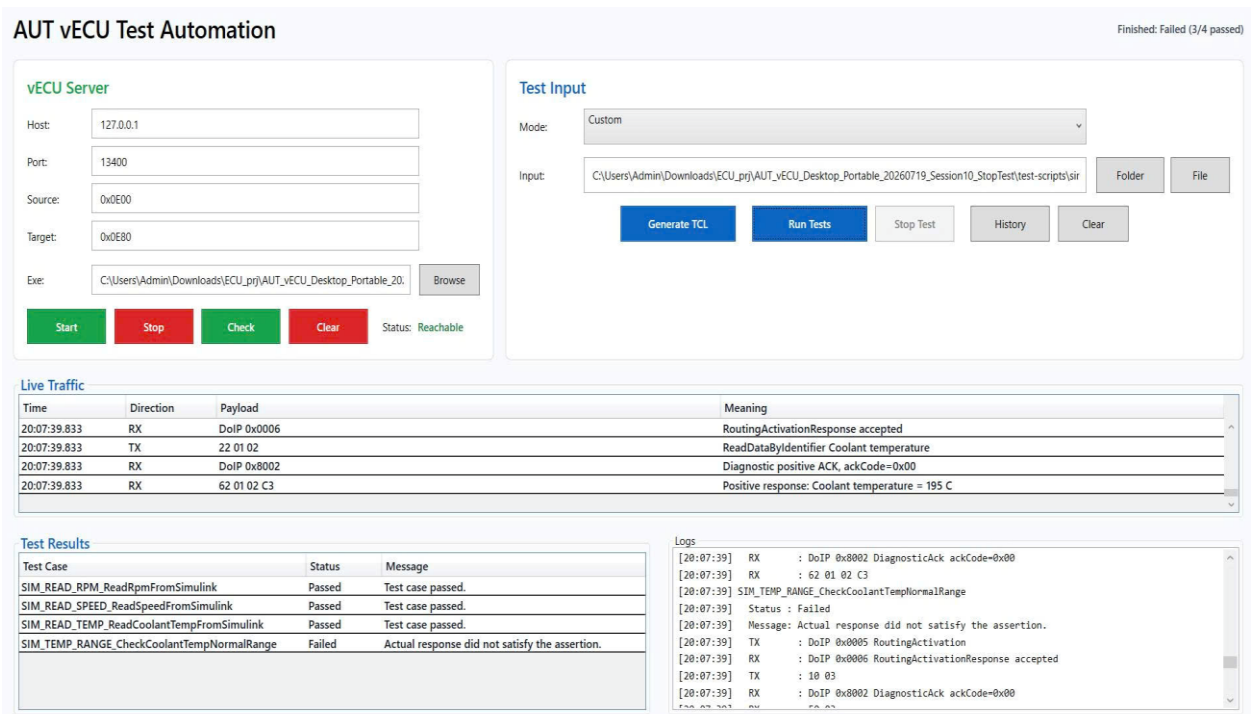


Figure 4.12: Automated UDS HTML Test Execution Report - 100% Passed

4.2 Automated CI/CD Regression Results

Continuous integration and automated regression testing are enforced through the declarative Jenkins pipeline, ensuring that any modifications to the core C source code or MATLAB integration scripts do not introduce functional regressions. Upon triggering a build, the Jenkins server automatically restores dependencies, compiles the .NET and C components in Release mode, executes unit tests, and boots the **vECU.exe** process in the background while monitoring TCP port 13400 for socket binding availability. Subsequently, the **run_tests.bat** script launches the Tcl/JSON test runner to inject predefined test sequences (VTC_01 to VTC_05). Execution logs, UDS response packets, and HTML test summaries are successfully captured and archived as build artifacts, providing a fully transparent, repeatable quality gate that verifies system stability without requiring manual hardware intervention.

Automated regression pipelines executing under Jenkins generate HTML Test Report artifacts upon completion. When all UDS test cases satisfy assertion criteria, the HTML dashboard reports a 100% success rate (PASSED: 4 Total, 4 Passed, 0 Failed).

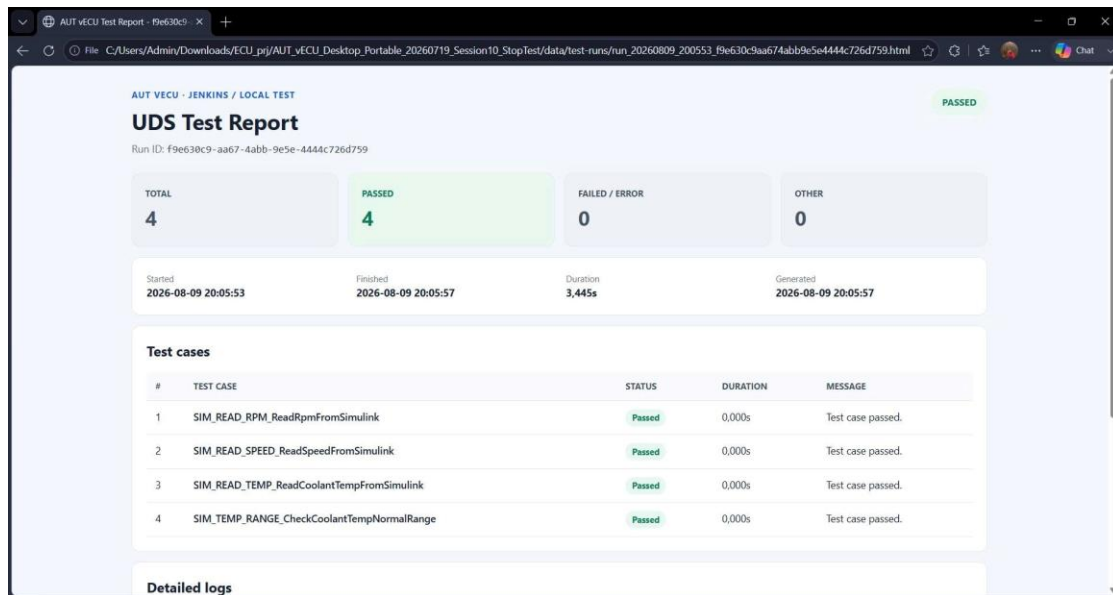


Figure 4.13: Automated UDS HTML Test Execution Report - Failed Execution Summary

In scenarios where assertion failures occur, the HTML dashboard flags the overall run as FAILED, listing the exact failing test case and associated assertion log for immediate developer diagnostic triage.

CHAPTER 5. CONCLUSION AND FUTURE WORK

5.1 Conclusion

The project demonstrates that an automated SIL diagnostic framework can provide a practical and repeatable validation environment for an engine-control vECU. The prototype integrates a C-based DoIP/UDS server with modeled AUTOSAR diagnostic responsibilities, a MATLAB/Simulink co-simulation loop, a WPF operator interface, and script-driven regression testing under Jenkins. The verification results show that the platform can execute the intended routing activation, session transition, seed-key security access, data writing, DTC management, and safety-inhibition scenarios without requiring a physical vehicle or HIL bench. This separation of the diagnostic application from the hardware-dependent test environment improves repeatability, supports safe fault injection, and creates a transparent CI/CD quality gate through archived test evidence.

5.2 Future Work

Future development should broaden the fidelity and scalability of the platform. Useful extensions include support for additional UDS services and diagnostic sessions, a more complete AUTOSAR configuration model, deterministic security algorithms suitable for test environments, and persistent diagnostic memory with richer freeze-frame records. The communication layer can be extended beyond the local loopback interface to support networked test benches and additional vehicle networks. Further work should also add larger data-driven regression suites, performance and robustness testing under concurrent connections, and a migration path from SIL validation to HIL or target-ECU integration. Additionally, the framework can be expanded to support Over-The-Air (OTA) software update simulations by integrating UDS reprogramming services (e.g., Request Download 0x34, Transfer Data 0x36) and a virtual bootloader

REFERENCES

- [1] International Organization for Standardization, Road Vehicles — Diagnostics over Internet Protocol (DoIP) — Part 2: Transport Protocol and Network Layer Services, ISO Standard 13400-2:2012, Aug. 2012.
- [2] AUTOSAR Development Partnership, Specification of Diagnostic Communication Manager, AUTOSAR Classic Release 4.4.0, SWS DCM, Nov. 2020.
- [3] AUTOSAR Development Partnership, Specification of Diagnostic Event Manager, AUTOSAR Classic Release 4.4.0, SWS DEM, Nov. 2020.
- [4] AUTOSAR Development Partnership, Specification of Function Inhibition Manager, AUTOSAR Classic Release 4.4.0, SWS FiM, Nov. 2020.
- [5] AUTOSAR Development Partnership, Specification of Diagnostic Extract, AUTOSAR Classic Release 4.4.0, SWS DEX, Nov. 2020.
- [6] AUTOSAR Development Partnership, Specification of Diagnostic Log and Trace, AUTOSAR Classic Release 4.4.0, SWS DLT, Nov. 2020.
- [7] J. Lindberg, "Security analysis of vehicle diagnostics using DoIP," M.S. thesis, Dept. Comput. Sci. Eng., Chalmers Univ. Technol., Gothenburg, Sweden, May 2011.
- [8] T. Axerot, "Research and development of a vehicle simulation platform: Design of a flexible and versatile architecture for vehicle simulation to satisfy the needs of diagnostic applications," M.S. thesis, Dept. Comput. Sci. Eng., Chalmers Univ. Technol., Gothenburg, Sweden, Jan. 2011.
- [9] A. Cheng, T. Chen, and D. Sun, "Design of configurable diagnostic test database based on UDS protocol," *Int. Core J. Eng.*, vol. 2, no. 10, pp. 116–123, Oct. 2016.
- [10] Z. Guo, K. Koufos, M. Dianati, and R. Woodman, "State-of-the-art virtualisation technologies for the centralised automotive E/E architecture," *Frontiers in Future Transportation*, vol. 6, Art. no. 1519390, Jan. 2025.

- [11] F. Adegbohun, A. von Jouanne, B. Phillips, E. Agamloh, and A. Yokochi, "High performance electric vehicle powertrain modeling, simulation and validation," *Energies*, vol. 14, no. 5, Art. no. 1493, Mar. 2021.
- [12] V. Tomar, A. Chitra, D. Krishnachaitanya, N. S. R. Rao, V. Indragandhi, and W. Raziasultana, "Design of powertrain model for an electric vehicle using MATLAB/Simulink," in *Proc. IEEE 2021 Innovations in Power and Advanced Computing Technologies (i-PACT)*, Kuala Lumpur, Malaysia, Nov. 2021, pp. 1–6.
- [13] A. Bhatti, "Model based design framework: Development of a hybrid supervisory controller for a P4 parallel hybrid vehicle," M.A.Sc. thesis, Dept. Mech. Mechatronics Eng., Univ. Waterloo, Waterloo, ON, Canada, 2021.
- [14] A. Khanna, "Full-vehicle model development of a hybrid electric vehicle and development of a controls testing framework," M.S. thesis, Dept. Mech. Eng., The Ohio State Univ., Columbus, OH, USA, 2016.
- [15] W. Jin, A. Orso, and T. Xie, "Automated behavioral regression testing," in *Proc. 3rd IEEE Int. Conf. Software Testing, Verification and Validation (ICST)*, Paris, France, Apr. 2010, pp. 137–146.
- [16] W. Pensec, V. Lapotre, and G. Gogniat, "Scripting the unpredictable: Automate fault injection in RTL simulation for vulnerability assessment," in *Proc. 27th Euromicro Conf. Digital System Design (DSD)*, Paris, France, Sep. 2024.
- [17] Y.-W. Huang, S.-K. Huang, T.-P. Lin, and C.-H. Tsai, "Web application security assessment by fault injection and behavior monitoring," in *Proc. 12th Int. Conf. World Wide Web (WWW)*, Budapest, Hungary, May 2003, pp. 148–159.
- [18] M. O. Esquerdo, "Continuous integration framework for automotive software platforms," Master's thesis, La Salle School of Telecommunications Engineering, Ramon Llull Univ., Barcelona, Spain, 2021.
- [19] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Reading, MA, USA: Addison-Wesley, 1995.

- [20] M. Fowler, Patterns of Enterprise Application Architecture. Boston, MA, USA: Addison-Wesley Professional, 2002.
- [21] Vector Informatik GmbH, MICROSAR Classic Product Catalogue, Vector Informatik GmbH, Stuttgart, Germany, 2023.
- [22] J. Wolf and J. Toennemann, MICROSAR Classic Safe Product Information, Vector Informatik GmbH, Stuttgart, Germany, 2023.
- [23] ASAM Task Force, ASAP Standard CAN Calibration Protocol (CCP), Version 2.1, Association for Standardization of Automation and Measuring Systems (ASAM), Feb. 1999.
- [24] J. Smart, Jenkins: The Definitive Guide. Sebastopol, CA, USA: O'Reilly Media, 2011.
- [25] L. Bass, I. Weber, and L. Zhu, DevOps: A Software Architect's Perspective. Boston, MA, USA: Addison-Wesley Professional, 2015.
- [26] A. Nathan, WPF 4.5 Unleashed. Indianapolis, IN, USA: Sams Publishing, 2013.
- [27] M. Smith, MVVM in Delphi: Architecting and Building Model View ViewModel Applications. Berkeley, CA, USA: Apress, 2020.
- [28] W. R. Stevens, B. Fenner, and A. M. Rudoff, Unix Network Programming, Volume 1: The Sockets Networking API, 3rd ed. Boston, MA, USA: Addison-Wesley Professional, 2003.
- [29] J. K. Ousterhout and K. Jones, Tcl and the Tk Toolkit, 2nd ed. Boston, MA, USA: Addison-Wesley Professional, 2009.
- [30] P. Ammann and J. Offutt, Introduction to Software Testing, 2nd ed. Cambridge, UK: Cambridge University Press, 2016.
- [31] ECMA International, The JSON Data Interchange Syntax, ECMA Standard 404, 2nd ed., Dec. 2017.
- [32] International Organization for Standardization, Information Technology — Programming Languages — C, ISO/IEC Standard 9899:2018 (C18), Jun. 2018.

